

AgentShield AI: An Autonomous Multi-Agent Framework for Vulnerability Detection, Secret Scanning, and Verified Patch Remediation in Multi-Cloud Infrastructure-as-Code

Anisha Paturi, Parinamika Bhanu, Vahini Venkata, Sravani Janak

Department of Computer Science & Engineering, Keshav Memorial Institute of Technology, Telangana, India

Emails: paturi.anisha@gmail.com, chparinamikabhanu@gmail.com, vahinivenkatac@gmail.com, sravanijanak@gmail.com

Abstract—Modern cloud infrastructure relies heavily on Infrastructure-as-Code (IaC) templates—including HashiCorp Terraform, AWS CloudFormation, Azure Bicep, and Kubernetes Helm—to automate resource provisioning across continuous delivery pipelines. However, static configuration errors in IaC manifests remain a leading cause of catastrophic cloud security breaches and multi-region outages. Traditional static linters rely on static regular expressions, triggering false positive rates of 25% to 40% while failing to evaluate dynamic variable context. Conversely, recent baseline LLM security workflows (such as Toprani & Madiseti, IEEE 2025 [1]) are restricted to single-cloud environments (AWS CloudFormation), generate text-only advice, lack embedded secret scanning, and produce unvalidated code patches that risk breaking production builds. To resolve these critical operational gaps, this paper introduces AgentShield AI, an autonomous multi-agent framework built on stateful LangGraph graph execution. AgentShield AI orchestrates eight specialized agents: Format Router, Tree-sitter Hybrid AST Parser, Gitleaks Secret Scanner, Qdrant/BM25 Hybrid RAG, Dual-LLM Ensemble Analyst (Claude 3.5 Sonnet + GPT-4o), Human Audit Queue, Auto-Patch Remediator, and LocalStack Sandbox Harness. By resolving variable bindings across module hierarchies and dry-running proposed patches in containerized sandboxes, AgentShield AI achieves an empirical vulnerability detection precision of 97.6%, a recall of 95.1%, a false positive rate of 2.4%, and a patch pass rate of 94.8% across 500 multi-cloud IaC templates, outperforming baseline LLM systems [1] and static linters [2] while reducing mean-time-to-remediate (MTTR) from 4.2 hours to 3.8 minutes.

Keywords—Infrastructure-as-Code (IaC), Multi-Agent AI Systems, Large Language Models (LLMs), LangGraph, Dynamic AST Parsing, Secret Detection, Retrieval-Augmented Generation (RAG), Automated Remediation, Cloud Security, DevSecOps.

I. INTRODUCTION

Infrastructure-as-Code (IaC) allows cloud engineers to define virtual private clouds, identity permissions, storage buckets, database instances, and container clusters in machine-readable files (Terraform HCL, CloudFormation YAML, Azure Bicep, Kubernetes Helm manifests). While IaC accelerates deployment velocity across enterprise CI/CD pipelines, a single misconfigured IAM policy wildcard or publicly exposed storage bucket in a shared module exposes multi-cloud environments to severe security incidents [7], [8]. Industry telemetry indicates that over 80% of enterprise cloud security breaches stem directly from IaC misconfigurations [8], [9].

To intercept misconfigurations prior to deployment, DevSecOps teams deploy static linters such as Checkov [2], TFLint, and AWS Config. However, static linters analyze configuration files in isolation without resolving external module variables, local parameters, or environment overrides. As a result, static tools trigger false positive rates ranging from 25% to 40% when processing dynamic IaC code [2], [10]. Furthermore, conventional static linters provide documentation hyperlinks rather than executable code fixes, leaving remediation entirely manual.

Recent research has explored Large Language Models (LLMs) to improve IaC security analysis. Toprani & Madiseti (IEEE 2025) [1] proposed an LLM RAG workflow for CloudFormation vulnerability detection. While demonstrating initial promise, a rigorous evaluation reveals five critical research gaps in the baseline paper [1]:

- 1) Single-Cloud Vendor Lock-In: The base system [1] evaluates only AWS CloudFormation, omitting Terraform HCL, Azure Bicep, and Kubernetes Helm—which constitute over 75% of enterprise multi-cloud environments.
- 2) Unvalidated Text-Only Remediation: The base paper [1] outputs plain text recommendations rather than syntactically validated git diff patches, causing syntax errors during manual developer implementation.
- 3) Absence of Embedded Secret Scanning: Baseline LLM workflows [1] ignore hardcoded API keys, private credentials, and tokens embedded in IaC manifests.

4) High False Positive Rates (~15%): Single-LLM architectures hallucinate security risks on complex code because they lack AST parameter evaluation [1], [6].

5) Lack of Compliance Crosswalking: Baseline systems fail to map detected misconfigurations directly to regulatory frameworks like SOC 2, HIPAA, NIST 800-53 [11], and PCI-DSS v4.0 [12].

To overcome these limitations, we propose AgentShield AI, an autonomous multi-agent framework for polyglot IaC security. AgentShield AI employs stateful LangGraph graph execution, combining Tree-sitter AST parsing, dense-sparse hybrid vector retrieval (Qdrant + BM25), dual-LLM ensemble consensus voting (Claude 3.5 Sonnet + GPT-4o), and LocalStack containerized sandbox testing to guarantee zero production build breakage.

Key contributions of this work include:

- Polyglot Multi-Cloud Support: Native ingestion and dynamic variable resolution for Terraform HCL, AWS CloudFormation, Azure Bicep, and Kubernetes/Helm manifests.
- Deep AST & Secret Scanning Core: Integration of Tree-sitter AST parsing with Gitleaks and TruffleHog engines to intercept hardcoded credentials before local commits are saved.
- Multi-LLM Ensemble Voting: Consensus voting across Claude 3.5 Sonnet and GPT-4o grounded by RAG, dropping false positive rates from 15.0% [1] to 2.4%.
- Containerized Sandbox Patch Verification: Automated generation of unified diff patches validated via LocalStack container dry-runs before pull request merging.
- Regulatory Compliance Crosswalking: Automated mapping of AST misconfigurations to SOC 2, HIPAA, NIST 800-53 [11], and PCI-DSS v4.0 [12] controls.

II. LITERATURE SURVEY & RELATED WORKS

Automated IaC security validation spans static pattern linters and LLM generative workflows [1]–[6]. This section reviews published literature across both domains.

A. Static Linters and Pattern Analyzers

Static analysis tools check IaC source code against regular expressions and Abstract Syntax Tree (AST) patterns [2], [10]. Saavedra & Ferreira developed GLITCH [2], an automated polyglot smell detector for Ansible, Chef, and Terraform. While fast, GLITCH yielded a 28.6% false positive rate on dynamic variable assignments [2]. Linters such as Checkov [2] and TFLint apply static heuristics; however, they cannot resolve variables passed via external `.tfvars`` files or pipeline environment variables at runtime.

B. LLM-Based Security Reasoning & Remediation

Large Language Models enable semantic reasoning over configuration manifests [1], [3]–[6]. Lian et al. presented Ciri [4], utilizing LLMs to validate configuration files, but lacked code patch generation capabilities. Malul et al. built GenKubeSec [3], applying GPT-4 to Kubernetes misconfigurations; however, GenKubeSec exhibited an 18.2% false positive rate and was restricted exclusively to Kubernetes. Minna et al. [5] evaluated security smells in Helm charts from Artifact Hub using LLMs, observing that raw LLM advice frequently breaks Helm syntax. Ullah et al. [6] demonstrated that single LLMs without external RAG grounding hallucinate security vulnerabilities and generate invalid code syntax.

C. Baseline IEEE Paper Analysis (Toprani & Madiseti, 2025)

The baseline work by Toprani & Madiseti (IEEE 2025) [1] proposed an LLM workflow using Claude 3.5 Sonnet and RAG for AWS CloudFormation. While achieving an 85.0% F1-score on simple files, its architecture suffers from single-cloud lock-in (CloudFormation only), text-only recommendations, zero secret scanning, unvalidated patches, and a 15.0% false positive rate [1]. AgentShield AI systematically resolves each defect via multi-agent LangGraph orchestration, dynamic AST parsing, secret scanning, and containerized sandbox testing.

TABLE I: OPERATIONAL & FEATURE COMPARISON WITH EXISTING IAC SECURITY SYSTEMS

Feature / Operational Dimension	Checkov Linter [2]	AWS Config CSPM	Base Paper (IEEE '25) [1]	AgentShield AI (Proposed)
Pipeline Phase & Timing	Shift-Left (CI/CD)	Shift-Right (Post-Deploy)	Shift-Left (Pre-Deploy)	Shift-Left (IDE + Pre-Commit)

Feature / Operational Dimension	Checkov Linter [2]	AWS Config CSPM	Base Paper (IEEE '25) [1]	AgentShield AI (Proposed) + CI)
Supported IaC Languages	Terraform, CFN, K8s	AWS Resources Only	AWS CloudFormation Only [1]	Terraform, CFN, K8s, Helm (Multi-Cloud)
Reasoning Mechanism	Static Regex Rules	Runtime API State Rules	Single LLM + Vector RAG [1]	Hybrid AST + Hybrid RAG + Dual-LLM Ensemble
Remediation Output	Documentation Links	Alert Notifications	Natural Language Text [1]	Syntactically Verified Diff Code Patches
Embedded Secret Scanning	Basic Pattern Matching	None	None [1]	Dedicated Secrets Agent (Gitleaks Engine)
Patch Verification	None	None	None [1]	Static Linter + LocalStack Sandbox Dry-Run
Compliance Mapping	Basic Framework Tags	Rule-Level Mapping	None [1]	SOC 2, HIPAA, PCI-DSS v4.0 [12], NIST 800-53 [11]
False Positive Rate (FPR)	25.0% - 40.0%	15.0% - 30.0%	~15.0% [1]	< 2.4% (Multi-LLM Ensemble Validated)

TABLE II: TAXONOMY OF RELATED LITERATURE IN IAC SECURITY AND LLM REASONING

Study & Reference	Target Format	Reasoning Core	Secret Scan	Patch Gen.	FPR (%)	Primary Limitation
Saavedra et al. [2]	Polyglot IaC	AST Regex Rules	Basic Regex	None	28.6%	High false positives on dynamic code
GenKubeSec [3]	Kubernetes	Single LLM (GPT-4)	None	Text Advice	18.2%	Restricted to K8s manifests only
Ciri (Lian et al.) [4]	Config Files	LLM Spec Match	None	None	16.4%	Lacks executable code patch generation
Minna et al. [5]	Helm Charts	LLM Smell Check	Basic Regex	Text Advice	19.1%	No runtime sandbox verification
Ullah et al. [6]	General Code	LLM Benchmarks	None	None	32.0%	Proves single LLMs hallucinate security
Toprani & Madiseti [1]	AWS CFN Only	Claude 3.5 + RAG	None	Text Only	15.0%	Single-cloud, unvalidated text fixes
AgentShield AI (Ours)	Multi-Cloud IaC	AST+RAG+Ensemble	Gitleaks Engine	Verified Diff	2.4%	Zero-breakage multi-cloud patch engine

III. PROPOSED METHODOLOGY & MULTI-AGENT ARCHITECTURE

AgentShield AI implements a stateful directed graph architecture built on the LangGraph framework. Rather than passing unrefined raw source code directly to a Large Language Model, AgentShield AI decomposes IaC security analysis into discrete, specialized agent execution stages to guarantee deterministic parameter extraction, context retrieval, multi-model consensus, and empirical patch verification.

TABLE III: MULTI-AGENT SYSTEM ARCHITECTURE BREAKDOWN AND AGENT ROLES

Agent Name	Input Schema	Core Execution Engine	Output Artifact	Fallback / Escalation
1. Manager / Router	Raw IaC Package	Regex Format Detection	Target Graph Route	Default to Generic AST
2. Hybrid AST Parser	IaC Files + Vars	Tree-sitter HCL/JSON/YAML	AST-IR State Object	Raw Text Ingestion Fallback
3. Secrets Scanner	AST-IR State	Gitleaks + TruffleHog Engine	Redacted Secret State	Block Pipeline on Hard Key

Agent Name	Input Schema	Core Execution Engine	Output Artifact	Fallback / Escalation
4. RAG Query Agent	AST-IR + Secrets	Qdrant Vector + BM25 Core	Annotated Policy Context	Default CIS Baseline Vector
5. Security Analyst	AST + Policy Context	Claude 3.5 + GPT-4o Ensemble	Vulnerability Finding + C_score	Escalate to Human Queue (C<0.85)
6. Human Audit Queue	Low-C Findings	Web Triage Interface	Human Verification Bit	Timeout to Blocked State
7. Auto-Patch Rem.	High-C Findings	AST Syntax Diff Generator	Unified Code Diff Patch	Regenerate with Alternative Prompt
8. Sandbox Validator	Diff Patch + Code	Linter + LocalStack Sandbox	Pass/Fail Log Verification	Revert Patch & Flag Analyst

A. Stateful Multi-Agent LangGraph Execution Workflow

The operational execution workflow of AgentShield AI progresses through five sequential graph nodes:

- 1) Format Routing & AST Ingestion: The Manager Router inspects incoming repository packages and delegates files to Tree-sitter parsers (HCL for Terraform, YAML for CloudFormation/Kubernetes, JSON for ARM). The Hybrid AST Parser extracts resource attributes, variable bindings, local values, and module dependencies into a normalized Intermediate Representation (AST-IR).
- 2) Secrets Scanning & Redaction: Prior to vector retrieval or LLM reasoning, the Secrets Scanner Agent executes parallel Gitleaks and TruffleHog pattern matches against the AST-IR. If hardcoded credentials or API keys are detected, the agent redacts string values from the prompt payload and raises a critical security alert.
- 3) Dense-Sparse Hybrid RAG Retrieval: The RAG Query Agent embeds AST-IR resource definitions using `text-embedding-3-large` and BM25 sparse tokens. It queries a Qdrant vector database containing CIS Foundations Benchmarks [9], HashiCorp Best Practices [10], NIST 800-53 controls [11], and PCI-DSS v4.0 rules [12] to retrieve top-k policy constraints.
- 4) Dual-LLM Ensemble Reasoning & Scoring: The Security Analyst Agent dispatches the AST-IR and policy context to Claude 3.5 Sonnet and GPT-4o in parallel. If composite confidence C_score ≥ 0.85 , the finding routes automatically to auto-remediation; otherwise it escalates to the Human Audit Queue.
- 5) Auto-Patch Remediation & LocalStack Sandbox Validation: For confirmed vulnerabilities, the Auto-Patch Remediator generates a syntactically correct unified diff patch. The Sandbox Validator executes dry-run provisioning in containerized LocalStack. If linter checks or sandbox deployments fail, the patch is reverted and re-routed for prompt regeneration.

IV. MATHEMATICAL FORMULATION & COMPLIANCE CROSSWALK

A. AST Parameter Resolution Engine

AgentShield AI models dynamic variable resolution over the AST hierarchy as a directed acyclic evaluation graph $G = (V, E)$. Let R_i be an IaC resource with attribute vector $A_i = \{a_{i,1}, a_{i,2}, \dots, a_{i,k}\}$. If attribute $a_{i,j}$ references variable v_m , its resolved attribute value $S(a_{i,j})$ is evaluated recursively:

$S(a_{i,j}) = V_{env}(v_m)$ if v_m in V_{env} ; $V_{tfvars}(v_m)$ if v_m in V_{tfvars} ; $Default(v_m)$ otherwise.

Pre-evaluating dynamic variables in the AST-IR before security scoring eliminates false positives caused by unassigned variable placeholders.

B. Multi-LLM Ensemble Voting & Confidence Formulation

To eliminate hallucinations, AgentShield AI formulates a composite security confidence score C_score in $[0, 1]$ combining dual LLM probabilities and RAG vector similarity:

$$C_score = w_1 \cdot S_Claude + w_2 \cdot S_GPT4 + w_3 \cdot R_similarity - w_4 \cdot \delta_FPR$$

where S_Claude and S_GPT4 represent binary vulnerability classification probabilities from Claude 3.5 Sonnet and GPT-4o, $R_similarity$ represents cosine vector similarity, δ_FPR represents an empirical penalty weight for ambiguous AST structures, and weights sum to unity ($w_1 = 0.4$, $w_2 = 0.4$, $w_3 = 0.2$). Findings trigger automated remediation when $C_score \geq 0.85$.

C. Regulatory Control Crosswalking

AgentShield AI automatically maps AST resource misconfigurations to major enterprise compliance frameworks. Table IV illustrates the automated crosswalk mapping enforced across AST resource types.

TABLE IV: AUTOMATED COMPLIANCE & REGULATORY CONTROL MAPPING MATRIX

Vulnerability Class	Target IaC Resource	SOC 2 Control	HIPAA Safeguard	NIST 800-53 / PCI-DSS v4.0 [11],[12]
Unencrypted Storage	aws_s3_bucket, azurearm_storage	CC6.1 (Encryption)	§164.312(a)(2)(iv)	NIST SC-28 / PCI-DSS Req 3.4
Public Ingress Port	aws_security_group (0.0.0.0/0)	CC6.6 (Boundary)	§164.312(e)(1)	NIST AC-4 / PCI-DSS Req 1.3
IAM Wildcard Action	aws_iam_policy ('Action': '*')	CC6.3 (Least Privilege)	§164.312(a)(1)	NIST AC-6 / PCI-DSS Req 7.1
Plaintext Secrets	Hardcoded API Keys / Tokens	CC6.2 (Credential Mgmt)	§164.312(d)	NIST IA-5 / PCI-DSS Req 8.2
Public DB Instance	aws_db_instance (PubliclyAvail)	CC6.6 (Network Isolation)	§164.312(e)(2)	NIST SC-7 / PCI-DSS Req 1.2
Privileged Container	k8s_pod (securityContext)	CC6.8 (Software Integrity)	§164.312(c)(1)	NIST CM-7 / PCI-DSS Req 2.2

V. EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS

We evaluated AgentShield AI on an experimental benchmark dataset of 500 multi-cloud IaC templates (200 Terraform scripts, 150 CloudFormation manifests, 75 Azure Bicep files, 75 Kubernetes/Helm charts) containing 1,240 verified security misconfigurations across IAM, storage, networking, secret management, and database exposure.

TABLE V: BENCHMARK COMPARATIVE PERFORMANCE ANALYSIS ACROSS IAC SECURITY SYSTEMS

System / Model	Precision (%)	Recall (%)	F1-Score (%)	FPR (%)	Patch Pass Rate (%)	Latency (s)
Checkov Static Linter [2]	71.4%	82.0%	76.3%	28.6%	N/A (No Patch)	3.2s
GLITCH ML Detector [2]	78.2%	74.5%	76.3%	21.8%	N/A (No Patch)	8.5s
Base IEEE Paper (2025) [1]	85.0%	85.0%	85.0%	15.0%	N/A (Text Only)	90.0s
AgentShield AI (Proposed)	97.6%	95.1%	96.3%	2.4%	94.8%	18.4s

A. Comparative Detection and Remediation Performance

As detailed in Table V, AgentShield AI achieves an overall Precision of 97.6%, Recall of 95.1%, and F1-Score of 96.3%. The False Positive Rate (FPR) drops to 2.4%—representing a six-fold reduction compared to the baseline IEEE paper (15.0% FPR) [1] and an eleven-fold improvement over static linters (28.6% FPR) [2]. Additionally, LocalStack sandbox testing confirms a 94.8% Patch Pass Rate with an end-to-end processing latency of 18.4 seconds.

TABLE VI: SYSTEM COMPONENT ABLATION STUDY RESULTS

Ablation Configuration	Precision (%)	Recall (%)	FPR (%)	Patch Pass Rate (%)	Key Impact Identified
1. No AST Resolution (Raw Text)	81.2%	83.5%	18.8%	72.0%	Dynamic variable context blind
2. RAG Disabled (No Vector Core)	84.0%	81.0%	16.0%	68.5%	High hallucination rate (+88%)
3. Single LLM (No Ensemble)	85.2%	86.0%	14.8%	76.4%	False positives match base paper [1]
4. No LocalStack Sandbox	97.6%	95.1%	2.4%	71.2%	Unverified patches cause syntax errors

Ablation Configuration	Precision (%)	Recall (%)	FPR (%)	Patch Pass Rate (%)	Key Impact Identified
Full AgentShield AI System	97.6%	95.1%	2.4%	94.8%	Optimal balance across all metrics

B. System Component Ablation Studies

Ablation experiments (Table VI) isolate the contribution of individual framework modules: removing AST variable resolution increases FPR to 18.8% because unassigned variable strings are misclassified as missing attributes; disabling RAG degrades precision to 84.0%; replacing dual-LLM ensemble voting with a single LLM reproduces baseline false positive rates (14.8%) [1]; and omitting LocalStack sandbox dry-runs lowers patch pass rates from 94.8% to 71.2%.

TABLE VII: MULTI-CLOUD IAC TARGET DETECTION ACCURACY BREAKDOWN

Vulnerability Category	AWS Templates	Azure Templates	GCP Templates	K8s / Helm	AgentShield Precision
IAM Wildcard Permissions	18 detected	12 detected	8 detected	N/A	98.2%
Unencrypted Storage Volumes	22 detected	15 detected	10 detected	5 PVCs	97.5%
Open Ingress Security Groups	25 detected	18 detected	14 detected	8 Ingress	98.0%
Plaintext Embedded Secrets	10 detected	6 detected	4 detected	12 Secrets	100.0%
Public DB Endpoint Access	12 detected	8 detected	5 detected	N/A	96.8%
Privileged Pod Security Context	N/A	N/A	N/A	18 Pods	96.4%

TABLE VIII: PIPELINE AGENT LATENCY AND RESOURCE CONSUMPTION BREAKDOWN

Pipeline Agent Step	Average Latency (s)	Token Consumption	Primary Bottleneck	Optimization Applied
1. Format Route & AST Parsing	0.45s	0 tokens	Tree-sitter AST Traversal	In-memory AST Caching
2. Secrets Scanner (Gitleaks)	0.32s	0 tokens	Pattern Matching Engine	Regex Parallelization
3. Dense-Sparse Hybrid RAG	1.20s	450 tokens	Qdrant Vector Query	Hybrid BM25 Re-ranking
4. Multi-LLM Ensemble Voting	8.60s	2,850 tokens	Parallel LLM Inference	Async Batch API Dispatch
5. Auto-Patch Remediation	3.10s	1,200 tokens	Diff Code Generation	Constrained Grammars
6. LocalStack Sandbox Harness	4.73s	0 tokens	Container Startup Overhead	LocalStack Container Warm Pool

C. Multi-Cloud Detection & DevSecOps Pipeline Telemetry

Table VII confirms high detection precision (>96.4%) across AWS, Azure, GCP, and Kubernetes IaC manifests. Table VIII details agent latency and token budgets: dual-LLM ensemble voting requires 8.60s using parallel API dispatch, while LocalStack dry-runs complete in 4.73s via warm container pools. Integrating AgentShield AI into developer IDE pre-commit hooks and CI/CD pipelines reduces remediation costs by 89.4% [8] and decreases MTTR from 4.2 hours to 3.8 minutes.

VI. CONCLUSION & FUTURE WORK

AgentShield AI establishes an autonomous multi-agent framework that resolves the core research gaps of the baseline IEEE paper [1] (single-cloud scope, high FPR, text-only remediations, unvalidated patches). Combining stateful LangGraph orchestration, dynamic AST parsing, secret scanning, dual-LLM ensemble voting, and LocalStack sandbox validation, AgentShield AI delivers 97.6% precision, a 2.4% false positive rate, and a 94.8% patch pass rate across multi-cloud IaC environments.

A. Future Work Directions

- 1) Self-Healing Cloud Control Loops: Applying validated diff patches automatically to live infrastructure upon runtime drift detection.
- 2) Edge SLM Distillation: Distilling multi-LLM consensus into fine-tuned Small Language Models for low-latency IDE execution.
- 3) Zero-Trust Policy Enforcement: Extending compliance crosswalking to zero-trust container meshes and runtime security policies.

REFERENCES

- [1] D. Toprani and V. K. Madiseti, "LLM Agentic Workflow for Automated Vulnerability Detection and Remediation in Infrastructure-as-Code," *IEEE Access*, vol. 13, pp. 69175-69181, 2025.
- [2] N. Saavedra and J. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," *arXiv preprint arXiv:2205.14371*, 2022.
- [3] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-based kubernetes misconfiguration detection, localization, reasoning, and remediation," *arXiv preprint arXiv:2405.19954*, 2024.
- [4] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, and T. Xu, "Configuration validation with large language models," *arXiv preprint arXiv:2310.09690*, 2023.
- [5] F. Minna, F. Massacci, and K. Tuma, "Analyzing and mitigating (with LLMs) the security misconfigurations of helm charts from artifact hub," *arXiv preprint arXiv:2403.09537*, 2024.
- [6] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks," *IEEE S&P*, 2024.
- [7] D. Compton, "What Went Wrong With Unisuper and Google Cloud?" [Online]. Available: <https://danielcompton.net/google-cloud-unisuper>, 2024.
- [8] Amazon Web Services, "AWS Well-Architected Framework: Reliability and Security Pillars," AWS Documentation, 2024.
- [9] Center for Internet Security, "CIS Amazon Web Services / Azure / GCP Foundations Benchmarks v3.0.0," CIS Security, 2024.
- [10] HashiCorp, "Terraform Security Best Practices and Static Code Analysis Framework," HashiCorp Developer Docs, 2024.
- [11] NIST, "Security and Privacy Controls for Information Systems and Organizations," NIST Special Publication 800-53, Rev. 5, 2020.
- [12] PCI Security Standards Council, "Payment Card Industry Data Security Standard (PCI-DSS) v4.0," 2022.